Graflar

Graf (Graph) Nedir ve C'de Nasıl Temsil Edilir?

- Graf'lar önceki pdf'de de belirtildiği üzere tree'lerin üst başlığıdır. Yani tree'lerdeki gibi belirli kurallara sıkıştırılmamıştır.
- Tree'lerdekine benzer olarak düğümler (node) ve onları bağlayan kenarlar (edge)'lerden oluşur. ancak tree'lerden farklı olarak her node birbirine istediği gibi bağlanabilir.

Peki Grafları Hafızada Nasıl Tutacağız?

1. **Komşuluk Matrisi:**
   a. 2 boyutlu bir dizi (`int graph[5][5]`) kullanılır.
   b. Eğer A ile B bağlıysa oraya 1 yazarsın, yoksa 0.
   c. Kodlaması aşırı kolaydır ancak çok fazla bellek harcar (sonuçta komşu olmayanlar için de ayrıca bilgi tutmak zorundasın.)
2. **Komşuluk Listesi:**
   a. Her düğümün, bağlı olduğu diğer düğümleri tuttuğu bir linked list vardır.
   b. Sadece varolan bağları tuttuğu için hafıza dostudur.

   Komşuluk Listesi Örneği:

```
#include <stdio.h>
#include <stdlib.h>

// Bu yapı, komşuluk listesindeki her bir elemanı (bağlant
ıyı) temsil eder
```

```
struct Node {
    int dest;           // Hangi düğüme gidiyor? (Örn: 2 n
umaralı düğüm)
    struct Node* next;  // Bir sonraki komşu kim?
};

// Bu yapı grafın kendisi.
// 'array' dediğimiz şey, her düğümün başlangıç pointer'ın
ı tutan bir dizi olacak.
struct Graph {
    int numVertices;        // Toplam düğüm sayısı
    struct Node** adjLists; // Pointer dizisi (Linked List
lerin başı)
    int* visited;           // BFS/DFS yaparken ziyaret ed
ildi mi diye tutacağız
};

// Yeni düğüm oluşturma (Linked List mantığı)
struct Node* createNode(int dest) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

// Graf oluşturma
struct Graph* createGraph(int vertices) {
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;

    // Düğüm sayısı kadar pointer dizisi oluşturuyoruz
    graph->adjLists = malloc(vertices * sizeof(struct Node
*));
    graph->visited = malloc(vertices * sizeof(int));

    for (int i = 0; i < vertices; i++) {
```

```c
        graph->adjLists[i] = NULL; // Başlangıçta kimse ki
mseye bağlı değil
        graph->visited[i] = 0;     // Kimse ziyaret edilme
di
    }
    return graph;
}

// Kenar Ekleme (Örn: 0 ile 1'i bağla)
void addEdge(struct Graph* graph, int src, int dest) {
    // Kaynaktan Hedefe (src -> dest) bağ ekle
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    // Eğer YÖNSÜZ (Undirected) graf ise, tam tersini de e
klemelisin (dest -> src)
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}
```

- C Dilinde Daha Detaylı Bir Graf İskeleti Örneğine Aşağıdaki Linkten Ulaşabilirisiniz:

https://github.com/Mertcangndn/ListNode-Workspace/blob/main/Graph/Basic-Graph-C/main.c

addEdge() Fonksiyonu Açıklaması:

1. `createNode(2)` : 2'yi yarat.

   Plaintext

   ```
   [2] (Boşta)
   ```

2. `newNode->next = adjLists[0]` : 2, kutunun gösterdiği eski elemana (1'e) bağlansın.

   Plaintext

   ```
           /-----> [1] -> NULL
   [2] ---/
   ```

3. `adjLists[0] = newNode` : Kutu artık yeni elemanı (2'yi) göstersin.

   Plaintext

   ```
   Kutu[0] ------> [2] -> [1] -> NULL
   ```

Dinamik Sayılı Graph Yapımı

Orijinali nasıldı?

```
HAFIZADAKİ GÖRÜNTÜ (GRAF)

[adjLists DİZİSİ]
+-------+
|  [0]  | ---------------------> [Node 1] -> [Node 2] -> NULL
+-------+      (0'ın komşuları)
|  [1]  | ---------------------> [Node 3] -> NULL
+-------+      (1'in komşuları)
|  [2]  | ---------------------> NULL
+-------+
   ^
   |
   Burası sabit! (En başta malloc ile 5 tane dedik, bitti.)
```

Şu an yapacağımız nasıl?

```
HAFIZADAKİ GÖRÜNTÜ (DOSYA SİSTEMİ)

     [ ROOT ]
        |
        | (firstChild - "İçimdeki ilk eleman")
        v
    [ ODEVLER ] ----------------------------> [ OYUNLAR ] -> NULL
        |           (nextSibling - "Kardeşim")
        |
        | (firstChild)
        v
  [ matematik.txt ] -> NULL
      (nextSibling)
```

```
📁 ROOT (En Dış Klasör)
│
├── 📁 ODEVLER      <--- (ROOT'un içinde)
│   │
│   └── 📄 matematik.txt  <--- (ODEVLER'in içinde)
│
└── 📁 OYUNLAR      <--- (ROOT'un içinde, ODEVLER ile aynı seviyede)
```

```c
typedef struct FileNode {
    char name[50];           // Dosya veya Klasör adı (Ör
n: "Odevler")
    int isFolder;            // 1 ise Klasör, 0 ise Dosya

    struct FileNode* parent; // Üst Klasör (cd .. yapınca
döneceğimiz yer)

    // --- BURASI KRİTİK ---
    // Bir klasörün içinde sadece İLK dosyayı tutarız.
    // Diğer dosyalar, o ilk dosyanın "kardeşi" (sibling)
olarak tutulur.
    // Tıpkı graf'taki "temp->next" mantığı gibi!

    struct FileNode* nextSibling; // İlk posta örneğineki
daireleri posta kutularını birbirine bağlı tutar
    struct FileNode* firstChild;  // Dairelerin posta kutu
larının içindeki kağıtlardır

} FileNode;
```

Graflarda Gezinme Algoritmaları

DFS (Depth First Search - Derinlemesine Arama)

- Başlangıç noktasından itibaren bir yola girer ve bitene kadar o yolda devam eder.
- Gezdiği her node'u visited dizisine kaydeder.
- Yolu bittiğinde bir önceki odaya döner ve diğer yoldan sonuna kadar devam eder (backtracking).

```
void DFS(Graph* graph, int start){  //start: hangi düğümden a
ramaya başlanacağıdır.

    // ADIM 1: Geldiğim yeri işaretle ve yazdır
    graph ->visited[start]=1;
    printf("%d",start);

    // ADIM 2: Komşulara bak
    Node* tempNode = graph->adjLists[start];
    while(tempNode){
        int neighborNode = tempNode->value;

        // ADIM 3: Eğer o komşuya daha önce girmediysen o yön
de git.
        if(graph->visited[neighborNode]==0){
            DFS(graph, neighborNode); //Rekürsif kısım
        }

        //Sonraki Komşuya Geç
        tempNode=tempNode->next;
    }
}
```

```Plaintext
 AHMET'İN KUTUSU       MEHMET'İN KAĞIDI        ALİ'NİN KAĞIDI
(adjLists[start])        (tempNode)             (tempNode->next)
+--------------+       +--------------+       +--------------+
|  POINTER     | ----> |  value: 1    | ----> |  value: 2    | --> NULL
+--------------+       |  (Mehmet)    |       |  (Ali)       |
                       +--------------+       +--------------+
```

BFS (Breadth First Search - Genişlemesine Arama)

- Başlangıç noktasından itibaren bulunduğu node'un komşularını sırayla gezer.
- Gezdiği her node'u visited dizisine kaydeder.
- Komşular bittiğinde komşularun komşularıyla devam eder.

```
void BFS(Graph* graph, int start){

    // ADIM 1: Önce visited listesini temizleyelim (DFS'ten k
alan izler silinsin)
    for (int i = 0; i < graph->totalNode; i++)graph->visited
[i] = 0;

    // --- KUYRUK KURULUMU (Manuel Queue) ---
    int queue[100]; // 100 kişilik bir bekleme sırası
    int front = 0;  // Sıranın başı (Gişe)
    int rear = 0;   // Sıranın sonu (Yeni gelenin gireceği ye
r)

    // ADIM 2: Başlangıç düğümünü sıraya al ve işaretle
    graph->visited[start]=1;
    queue[rear] = start;
    rear++; //Sıra bir kişi uzadı

    // ADIM 3: Sırada bekleyen hiçbir node kalmayana kadar de
vam et.
```

```
    while(front<rear){

        //Sıradakini çağırma (dequeue)
        int currentNode = queue[front];
        printf("%d->",currentNode);
        front++; //Sıra ilerledi

        // Bu elemanın komşularını bul ve sıraya ekle
        Node* tempNode = graph->adjLists[currentNode];

        while(tempNode){
            int adjStart = tempNode->value;

            //Eğer komşu daha önce ziyaret edilmediyse
            if(graph->visited[adjStart] == 0){
                graph->visited[adjStart] = 1;  //ziyaret edil
di diye değiştir.
                queue[rear] = adjStart; //sıraya sok
                rear++;
            }
            tempNode = tempNode->next;
        }
    }
    printf("NULL");
    printf("\n");
}
```